

SOLVENTA: System Design

Arquitectura Cloud-Native para Aseguradora Digital

****Proyecto:** MISW4501 Proyecto Final**

****Caso:** Solventa - Aseguradora Digital de Finanzas Abiertas**

****Enfoque:** Latencia, Escalabilidad, Disponibilidad, Seguridad, Facilidad de Modificación e Integración**

****Versión:**** 1.0 (Diseño Preliminar)

TABLA DE CONTENIDOS

1. [Visión General](#1-visión-general)
2. [Decisiones Arquitectónicas Clave](#2-decisiones-arquitectónicas-clave)
3. [Modelo de Componentes](#3-modelo-de-componentes)
4. [Flujos de Datos Críticos](#4-flujos-de-datos-críticos)
5. [Estrategia de Escalamiento](#5-estrategia-de-escalamiento)
6. [API Design (Web, Móvil, Partners)](#6-api-design-web-móvil-partners)
7. [Datos y Consistencia](#7-datos-y-consistencia)
8. [Seguridad y Cumplimiento](#8-seguridad-y-cumplimiento)
9. [Resiliencia y Disponibilidad](#9-resiliencia-y-disponibilidad)
10. [Plan de Experimentos](#10-plan-de-experimentos)

1. VISIÓN GENERAL

1.1 Principios Rectores

<https://www.pearsoncmg.com/api/v1/print/healthcare/9780134160199>

■ ARQUITECTURA: MODULAR ORIENTADA A EVENTOS (MOE) ■

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116

- • Core síncrono para cotización embebida (latencia crítica) ■
- • Event-driven para perfilamiento, siniestros, analítica ■
- • APIs versionadas e independientes por canal ■
- • Escalamiento selectivo (no distribuido innecesariamente) ■
- • Datos locales por dominio + caché compartida ■
- • Cloud-first con multi-zona y multi-región ■



1.2 Conceptos Clave

Concepto	Significado
-----	-----
Core	Monolito modular que maneja cotización, suscripción, emisión

■ ■ RATING & ■ ■ UNDERWRITING ■ ■ POLICY ■ ■

■ ■ PRICING ■ ■ & DECISIONS ■ ■ LIFECYCLE ■ ■

■ ■ ■ ■ ■ ■ ■ ■

■ ■ • Actuarial ■ ■ • Auto rules ■ ■ • Emissions ■ ■

■ ■ • Open Fin. ■ ■ • Manual path ■ ■ • Changes ■ ■

■ ■ • Real-time ■ ■ • Explain. ■ ■ • Renewals ■ ■

[illegible]

■ ■

[illegible]

■ ■ CLAIMS ■ ■ COLLECTIONS ■ ■ IDENTITY ■ ■

■ ■ & PAYOUTS ■ ■ & BILLING ■ ■ & KYC ■ ■

Page 10 of 10

■ ■ • Intake ■ ■ • Premium ■ ■ • Onboarding ■ ■

■ ■ • Evaluation ■ ■ collection ■ ■ • Verification ■ ■

■ ■ • Automation ■ ■ • Payouts ■ ■ • Data mgmt ■ ■

- **Aplicación Core:** Contenedor escalable en Kubernetes, multi-instancia
- **Event Stream:** Separado en brokers externos (Kafka/Pulsar)
- **Batch Jobs:** Workloads aislados en Fargate/Spot instances
- **Base de Datos:** Compartida (PostgreSQL) con caché (Redis) enfrente

ARQUITECTURA DE DESPLIEGUE:



333






























4444

■ EKS - Solventa Core (Kubernetes) ■ ■

[illegible]

■ ■ Pod 1 ■ Pod 2 ■ Pod 3 ■ ... ■ ■ ■

■ ■ Solventa ■ Solventa ■ Solventa ■ ■ ■ ■

■ ■ (500m) ■ (500m) ■ (500m) ■ ■ ■ ■

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

4444

■ ■ HPA: 10-100 pods (p95 < 250ms) ■ ■ ■



























333

■ MSK (Kafka) - Event Stream ■ ■

■ ■■ events.policies (3 particiones) ■ ■

■ ■■ events.claims (5 particiones) ■ ■

■ ■■ events.profiles (10 particiones) ■ ■

■ ■■ events.telemetry (unlimited) ■ ■

333

■ RDS PostgreSQL (Multi-AZ) ■ ■

■ ■■ Master (us-east-1a) ■ ■

■ ■■ Standby (us-east-1b) ■ ■

■ ■ ■ Read replicas (analítica) ■ ■

11111

■ ElastiCache Redis ■ ■

Cache layer (perfil, sesiones)

■ ■■ Session store (mobile offline) ■ ■

3333

■ Fargate Batch Jobs ■ ■

■ ■ ■ Reprocessing profiles (Spark) ■ ■ ■

■ ■ ■ Analytics pipelines ■ ■ ■

■ ■■ Nightly reconciliation ■ ■

...

2.3 COMUNICACIÓN E INTERACCIÓN ENTRE COMPONENTES

Decisión: Híbrida Síncrona + Asíncrona (Bounded Contexts)

****Opción Elegida:****

FLUJO POR TIPO DE TRANSACCIÓN:

A) COTIZACIÓN EMBEBIDA (Síncrona end-to-end)



B) SUSCRIPCIÓN + EMISIÓN (Síncrono → Asíncrono)



C) SINIESTRO ASISTIDO (Síncrono + Async Workflow)



■ Cliente reporta [mobile] → Core.createClaim() [< 5s] ■



■ Síncrono: ■

■ 1. Validate policy [cache] ■

■ 2. Store claim intake [DB] ■

■ 3. Trigger evaluation [async event] ■

■ 4. Response: {claim_id, next_steps} ■



■ Asíncrono (Workflow): ■

■ Event: ClaimCreated → Eval service ■

■ → Check eligibility [2h] ■

■ → Approve/Deny [human review if needed] ■

■ → Pay (if approved) [1d] ■

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464

11

■ [evento_id] → enriched_profile (cache) + policy_rules ■

■ → CreatePayment(claim_id, amount) [sync via ledger] ■

■ → Event: ClaimResolved → Metrics ■

■ ■

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved.

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116

11

■ ■

■ 1. Fetch Open Finance signals [100ms - timeout] ■

■ 3. Merge with cached profile [10ms] ■

■ 5. Generate personalized offer [50ms] ■

□ □

■ - Event: OfferCreated → Analytics ■

■ - Audit trail → Compliance log ■

■ - Cache for 15min ■



****Patrones:****

| Patrón | Cuándo | Ventaja | Riesgo |

|-----|-----|-----|-----|

| ****Request/Response**** | Cotización, perfilamiento, consultas | Consistencia inmediata | Latencia, acoplamiento |

| ****Event Stream**** | Póliza emitida, siniestro aprobado, pagos | Desacople, escalabilidad | Eventual consistency |

| ****Saga Distribuido**** | Flujos multi-paso (suscripción) | Transacciones distribuidas | Complejidad |

| ****CQRS (opcional)**** | Analítica + reporte | Escala lecturas | Complejidad futura |

2.4 DATOS Y CONSISTENCIA

Decisión: Datos Locales por Dominio + Caché Compartida

****Opción Elegida:****

MODELO DE DATOS POR DOMINIO:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034

■ CORE DATABASE (PostgreSQL) ■

■ (Single source of truth para todo) ■

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

111

■ Schema: solventa_core ■

■ ■■ customers (PII cifrada) ■

■ ■■ policias (pólizas, estado, auditoría) ■

■■■ coverage_selections (coberturas elegidas) ■■

■ ■■ claims (siniestros, pagos) ■

■ ■■ transactions (pagos, débitos - ledger) ■

■ ■■ quotes (cotizaciones históricas) ■

■ ■■ partner_integrations (partner API keys, quotas) ■

■ ■■ audit_log (200M+ filas/año, archivado) ■

■■■ compliance_events (Open Finance, habeas data) ■■■

■ ■ ■ system_settings (rules, flags, configuración) ■

■ ■

■ Estrategia de indexación: ■

■ ■ ■ PK + FKs normales ■

■ ■ ■ Index en: customer_id, policy_id, status, created_at ■

■ ■ ■ BRIN para audit_log (time-series) ■

■ ■■ GIN para búsquedas full-text (claims description) ■

Hash index para consentimientos (Open Finance)

■ ■

■ Particionamiento (horizontal): ■

■■■ policies: por año (policies_2026_q1, _q2, ...) ■■

■ ■ ■ claims: por mes (claims_2026_01, _02, ...) ■

■■■ audit_log: por semana (archive old) ■■

■ ■ ■ transactions: por day (ledger, never delete) ■

22

123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100

■ REDIS CACHE (ElastiCache, Multi-AZ) ■

■ (Consistencia eventual, fallback en caso de caída) ■

■ ■

■ Estrategia TTL: ■

```
■ ■■ profile:{customer_id} ■
```

■ ■ ■■ Risk score, Open Finance signals, Open Data ■

■ ■ ■■ TTL: 15 min (regenerable) ■

■ ■ ■■ Invalidate on: customer_update, Open Finance event ■

333

■ ■■ policy:{policy_id} ■

- ■ ■ ■ Estado, cobertura, vigencia ■
- ■ ■ ■ TTL: 1h (read-heavy) ■
- ■ ■ ■ Invalidate: policy_change, renewal, cancellation ■
- ■ ■ ■
- ■ ■ ■ quote_request:{quote_id} ■
- ■ ■ ■ Precio, factores de riesgo, timestamp ■
- ■ ■ ■ TTL: 15 min (válido para cotización) ■
- ■ ■ ■ Read-through: generate if missing ■
- ■ ■ ■
- ■ ■ ■ session:{session_id} ■
- ■ ■ ■ User context, auth token, device info ■
- ■ ■ ■ TTL: 24h (web), 7d (mobile remember-me) ■
- ■ ■ ■ Secure: encrypted, HTTPS only ■
- ■ ■ ■
- ■ ■ ■ open_finance_consent:{customer_id} ■
- ■ ■ ■ Permisos actuales, revocación, estado ■
- ■ ■ ■ TTL: 5 min (compliance crítico) ■
- ■ ■ ■ Invalidate: revocation event ■
- ■ ■ ■
- ■ ■ ■ reference_data: ■
- ■ ■ ■ coverage_types, risk_factors, rules ■
- ■ ■ ■ TTL: 24h (o event-driven invalidation) ■
- ■ ■ ■ Preload en startup ■
- ■ ■ ■
- ■ ■ ■ bloom_filter:{entity_type} ■
- ■ ■ ■ Quick negative check (fraud, blocked lists) ■
- ■ ■ ■ TTL: 5 min ■
- ■ ■ ■ Fallback to DB if not found ■
- ■ ■ ■
- ■ ■ ■ Invalidation Strategy: ■
- ■ ■ ■ Proactivo: Customer update → Redis.delete(profile:*) ■
- ■ ■ ■ Reactivo: Cache-aside pattern (read-through) ■
- ■ ■ ■ Eventual: TTL natural expiry ■

■ ■■ Eventos: PolicyIssued → invalidate session + profile ■

■ ■

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

[illegible]

■ DATA LAKE / ANALYTICS (S3 + Athena) ■

■ (Eventual consistency, batch) ■

[illegible]

■ ■

■ s3://solventa-datalake/ ■

raw/

■ ■ ■ ■ events/ (Kafka → Kinesis Firehose → Parquet) ■

■■■■ db_snapshots/ (nightly PostgreSQL export) ■■

■ ■ ■ external/ (Open Data sources) ■

333

■ ■■ processed/ ■

customer_profiles/ (daily aggregate)

■ ■ ■■ policy_analytics/ (claims, payments, exposure) ■

■ ■ ■ fraud_signals/ (ML features) ■

■ ■ ■

reports/

regulatory_dashboards/ (SFC, habeas data)

■ ■ ■ business_metrics/ (KPIs, SLA tracking) ■

22

■ Partición: s3://.../yyyy/mm/dd/hh/ (CloudFront + Athena) ■

■ Format: Parquet (comprimido, schema evolution) ■

■ SLA: 1h lag desde evento hasta queryable ■

2

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

****Estrategia de Consistencia:****

| Recurso | Garantía | Mecanismo | SLA |

A horizontal number line with four equal segments, each containing a dashed line for a digit.

Policy (estado)	Strong	DB + cache invalidation	<1s
---------------------	--------	-------------------------	-----

Profile (riesgo)	Eventual	Cache TTL 15min	15min
----------------------	----------	-----------------	-------

Transactions (dinero)	Strong	Ledger append-only	<1s
---------------------------	--------	--------------------	-----

| ****Analytics/reporting**** | Eventual | Batch nightly | +24h |

****Audit trail**** | Strong | Immutable log, tamper-proof | <5min |

****Evolución Futura (si crece):****

- `Event Sourcing` para pólizas (audit + temporalidad)

- `CQRS` para analítica (desacople lecturas/escrituras)

- `Saga Pattern` para transacciones distribuidas

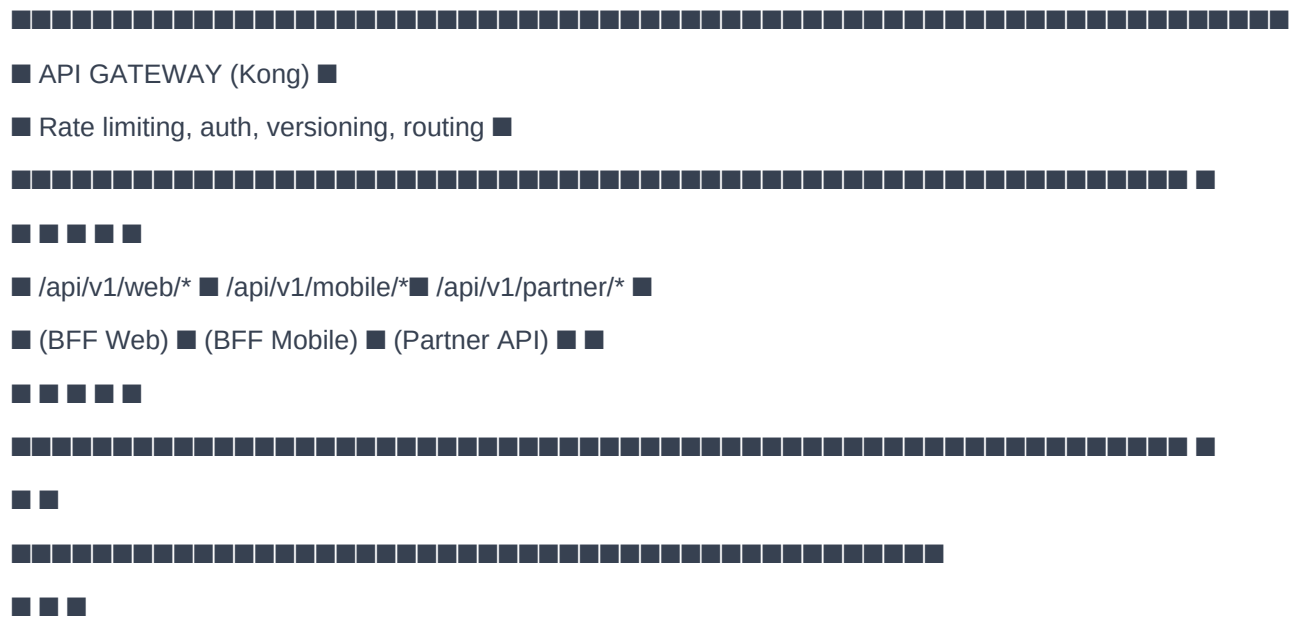
...

2.5 EXPERIENCIAS MULTICANAL: WEB, MÓVIL, PARTNERS

Decisión: APIs Versionadas + Adaptadores por Canal

****Opción Elegida:****

ARQUITECTURA MULTICANAL:



GET /policies/:id

■ ■ Response: { policy, coverage, premium, status, actions }

■ ■ Cache: Redis 1h (invalidate on change)

■ ■ Fallback: DB if cache miss

POST /claims

■ ■ Request: { policy_id, claim_type, description, attachments }

■ ■ Response: { claim_id, status, next_steps }

■ ■ Mobile: local indexedDB, sync on network

POST /profiles/:customer_id/mortgage-offer

■ ■ Request: { credit_amount, term_months }

■ ■ Response: { offer_id, price, coverage, rating_explanation }

■ ■ Cache: 15 min (per-profile)

■ ■ Timeout strategy: cache + fallback offer

=== BFF WEB (Aplicaciones, workflows) ===

GET /web/dashboard

- ■ ■ Compuesto: policies summary + claims status + actions
- ■ ■ Caché: server-side (Redis) + client (SPA memory)
- ■ ■ Orchestration: BFF
- ■ ■ Versioning: web app controls (no breaking changes)

POST /web/wizard/quote-and-subscribe

- ■ ■ Multi-step form: quote → accept → subscribe → pay
- ■ ■ State: session (Redis), submitted events (async)
- ■ ■ Transactional consistency: session-level

=== BFF MOBILE (Offline-first, push notifications) ===

GET /mobile/policies

- ■ ■ Compact: { id, cover_type, premium, expiry, status }
- ■ ■ Stored locally: IndexedDB (user consent)
- ■ ■ Sync: on network reconnect (conflict resolution: server wins)
- ■ ■ Fallback: cached content if offline

POST /mobile/claims/create-with-evidence

- ■ ■ Request: { policy_id, photos[], location, description }
- ■ ■ Storage: local queue (SQLite) → sync on connect
- ■ ■ Push notification: claim_status updates
- ■ ■ Offline: allow draft, sync + notify on reconnect

POST /mobile/biometric-auth

- ■ ■ Request: { biometric_data, session_token }
- ■ ■ Response: { access_token (24h), refresh_token (7d) }
- ■ ■ Security: face/fingerprint + PIN fallback

=== PARTNER API (Embedded Insurance) ===

POST /partners/v1/quotes

- ■ Authentication: OAuth2 + API key rotation
- ■ Authorization: scope-based (read_quotes, issue_policies)
- ■ Request: { partner_id, customer_id, coverage_type, amount }
- ■ Response: { quote_id, price, expires_at }
- ■ Ratelimit: 1000 req/min, burst 2000
- ■ SLA: p95 < 250ms

POST /partners/v1/policies

- ■ Issuing: pre-approved quotes only
- ■ Response: { policy_id, policy_number, effective_date }
- ■ Idempotency: idempotency-key header (24h retry window)
- ■ Webhook: policy_issued event (async confirmation)

GET /partners/v1/claims/:claim_id

- ■ Query: status, payment_status, documents
- ■ Authorization: partner that issued the policy only

=== VERSIONING STRATEGY ===

Breaking Changes:

- ■ Major version (v2) → deprecated v1 for 6 months
- ■ Parallel operation: both active during transition
- ■ Client routing: header Accept-Version: v1 | v2
- ■ Fallback: default to stable version

Non-breaking:

- ■ Same version, additive fields
- ■ Client ignores unknown fields (forward-compatible)
- ■ New functionality: feature flags (internal)

3. MODELO DE COMPONENTES

3.1 Componentes de Infraestructura

Componente	Tecnología	Justificación	Alternativa
----- ----- ----- -----			
Orquestación	EKS (Kubernetes)	Escalamiento automático, multi-zona nativa	Fargate (serverless, pero con latencia)
Event Stream	MSK (Kafka)	Throughput masivo, replayable, partitionable	SQS (simple, pero menos flexible)
Database	RDS PostgreSQL Multi-AZ	ACID, JSONB, funciones, comunidad	DynamoDB (serverless, pero schema-less)
Cache	ElastiCache Redis	Extremadamente rápido, clustering, pub/sub	Memcached (simple, no clustering)
Batch	Fargate + Spark	Elasticidad, Spot instances baratas	EC2 (caro si siempre encendido)
Object Storage	S3	Durabilidad 99.999%, Data Lake	EBS (volúmenes, peor para analytics)
API Gateway	Kong (on EKS)	Control fino, plugins, versionado	AWS API Gateway (simple, limitado)
Search	OpenSearch (optional)	Full-text en claims, audit trail	DB queries (más lento con volumen)

| ****Observability**** | Prometheus + Grafana + Jaeger | Open source, trace distribuido, control | CloudWatch (vendor lock-in) |

3.2 Servicios Core (En Monolito Modular)

// Pseudocódigo: estructura de módulos

package com.solventa.core;

// Dominio 1: Rating & Pricing (Cotización)

domain.rating {

RatingEngine {

calculate(customerId, coverageType, amount)

-> Quote {

// Obtiene profile (cache)

// Consulta Open Finance (timeout 120ms)

```
// Aplica reglas actuariales
```

```
// Retorna precio
```

```
}
```

```
}
```

```
OpenFinanceAdapter {
```

```
// Integración con proveedores Open Finance
```

```
// Timeout pattern, fallback a perfil cached
```

```
// Explicabilidad: qué datos se usaron
```

```
}
```

```
}
```

```
// Dominio 2: Underwriting (Suscripción)
```

```
domain.underwriting {
```

```
UnderwritingEngine {
```

```
decide(quoteId, customerSignal)
```

```
-> Decision {
```

```
// Auto-rules: accept/reject/refer
```

```
// Logs todas las reglas aplicadas (compliance)
```

```
}
```

```
}
```

```
ManualReviewWorkflow {
```

```
// Para casos ambiguos, UI para actuarios
```

```
// Audit trail completo
```

```
}
```

```
}
```

```
// Dominio 3: Policy Lifecycle
```

```
domain.policy {
```

```
PolicyService {
```

```
issue(quoteId, decision)
```

```
-> Policy {
```

```
// Emite póliza, genera PDF
```

```
// E-signature, envía email
```

```
// Event: PolicyIssued (async)
```

```
}
```

```
renew(policyId, newTerms)
```

```
modify(policyId, changes)
```

```
cancel(policyId, reason)
```

```
}
```

```
// Eventos publicados
```

```
events {
```

```
PolicyIssued { policyId, customerId, premium }
```

```
PolicyModified { policyId, changes }
```

```
PolicyCancelled { policyId, refund }
```

```
}
```

```
}
```

```
// Dominio 4: Claims & Payouts

domain.claims {
  ClaimService {
    create(policyId, description, attachments)
    -> Claim {
      // Valida póliza vigente
      // Inicia evaluación (async workflow)
      // Event: ClaimCreated
    }

    evaluate(claimId, evidence)
    approve(claimId, amount)
    pay(claimId, bank_details)
  }

  AutomaticClaimProcessor {
    // Para eventos paramétricos (IoT/Telemetry)
    // Kafka Streams → auto-decide → auto-pay
  }
}

// Dominio 5: Identity & KYC

domain.identity {
  KYCService {
    onboard(customer, documents)
    -> Customer {
      // Verificación identidad (proveedor)
      // AML checks (listas restrictivas)
      // Habeas data compliance
    }

    verify(biometric_data)
  }
}
```

```
ConsentManager {  
  // Open Finance: grant/revoke, audit trail  
  // Habeas data: qué datos se usan  
}  
}  
  
// Dominio 6: Profiling & Personalization  
domain.profile {  
  EnrichmentService {  
    enrich(customerId, consentedDataTypes)  
    -> EnrichedProfile {  
      // Open Finance: payment behavior, income stability  
      // Open Data: location, health, climate risk  
      // Merge with cached profile  
      // Cache result (15 min TTL)
```

```
}  
}
```

```
// Usa: Redis cache + eventual consistency  
}
```

```
// Dominio 7: Partner Integration  
domain.partner {  
  EmbeddedInsuranceAPI {  
    // REST API versionada  
    // Authentication: OAuth2 + API key  
    // Endpoints: POST /v1/quotations, POST /v1/policies  
  }  
}
```

```
PartnerManagement {  
  // Quotas, rate-limiting, API key rotation  
  // SLA tracking  
}  
}
```

```
// Dominio 8: Compliance & Audit  
domain.compliance {  
  AuditLog {  
    // Inmutable log de todas las transacciones  
    // Requerido para SFC, habeas data  
    // Particionado por fecha  
  }  
}
```

```
RuleEngine {  
  // Reglas de fraude (pattern matching)  
  // Alertas en tiempo real  
}  
}
```

```
// Servicios Transversales
```

```
shared {  
  AuthenticationService {  
    // JWT + OAuth2  
    // MFA para operadores  
  }  
  
  CacheService {  
    // Wrapper de Redis  
    // Patterns: cache-aside, invalidation  
  }  
  
  EventBus {  
    // Publisher → MSK (Kafka)  
    // Subscribers: Analytics, Notification, CRM  
  }  
}
```

```

ObservabilityService {
// Logging: structured (JSON)
// Metrics: Prometheus
// Tracing: Jaeger (distributed tracing)
}
}

---
```

4. FLUJOS DE DATOS CRÍTICOS

4.1 Flujo: Cotización Embebida (Síncrono, < 250ms p95)



■ POST /api/v1/quotations handler ■

■ 1. Auth + rate-limit check ■

■ 2. Input validation ■

■ 3. Get or create quote_request ■

■ 4. Call RatingEngine ■

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

t=5ms ■ RatingEngine.calculate(...)

[illegible]

■ RATING ENGINE (in-process) ■

11

■ A. Fetch profile from cache ■

```
■ Redis.get("profile:{cust_id}") ■
```


[illegible]

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

■ IoT SENSOR / Telemetry Provider ■

■ "Flight UA123 delayed 3+ hours" ■

■ [2026-01-15 14:30:00 UTC] ■

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464



t=0ms ■ Event published to Kafka

■ Topic: events.telemetry

- { flight_id, airline, delay_minutes, timestamp }

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10984.

■ KAFKA BROKER (MSK) ■

■ ■■ 10 partitions (parallelism) ■

■ ■■ Replication factor: 3 ■

■ ■ ■ Retention: 7 days ■

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

t=1ms ■ Event persisted

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

■ KAFKA STREAMS PROCESSOR ■

■ (Solvent embedded in Core) ■

11

■ Consumer: consumer-group-flight-delay■

■ State store: customer-flight-map ■

111

■ OnEvent: ■

```

    Deserialize { flight_id, delay }

```

■ ■■ Query state: policies with this ■

■ ■ flight? (enrichment lookup) ■

- ■ ■ For each policy: ■

■ ■ ■■ Load coverage rules (cache) ■

■■■■ flight_delay_coverage active? ■■

■ ■ ■■ Delay threshold met? (>3h) ■

■ ■ ■■ Generate claim_amount: ■

■ ■ ■ (coverage = \$500 delay) × ■

■ ■ ■ (delay_hours / 3) ■

■ ■ ■■ Emit: ClaimEventTriggered ■

11

123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100

t=50ms ■ Event published: ClaimEventTriggered

■ Topic: events.claims

```
■ { policy_id, claim_id, amount, reason: "AUTO_FLIGHT_DELAY" }
```


■ ■■ idempotency_key ■

t=120ms ■ Customer receives push notification



■ Notification: "Claim approved: \$50" ■

■ "Recent Transactions" ■



```
claims.claim_id = "CLM-2026-001234"
```

```
■ ■■ type = "PARAMETRIC"
```

■ ■ ■ amount = \$50

```
■■■■ created_at = 2026-01-15 14:30:50
```



■■ transactions.transaction_id = "TXN-001234"

■■ claim_id = "CLM-2026-001234"

■■ amount = \$50

■■ status = "COMPLETED"

■■ provider_ref = "PROV-12345678"

■■ created_at = 2026-01-15 14:31:10

TIMELINE:

■■ 0ms: Evento telemetry generado

■■ 1ms: Kafka recibe

■■ 50ms: Kafka Streams procesa, emite ClaimEventTriggered

■■ 100ms: Ledger service completa pago, emite PaymentInitiated

■■ 120ms: Notificación enviada

■■ 300ms: Cliente recibe push notification

TOTAL: < 5 minutos SLA ✓

CONTINGENCIAS:

■■ Kafka down → events buffered, procesa cuando recupera

■■ Payment provider timeout (120ms) → retry con backoff exponencial

■■ Customer account invalid → dead letter queue, manual review

■■ Duplicate events (same flight_id x 2 within 30s) → idempotency key

5. ESTRATEGIA DE ESCALAMIENTO

5.1 Escalamiento Horizontal

ESCENARIO 1: Campaña masiva de socio (100x tráfico)

■■ Pico: 500 cot/min → 50,000 cot/min

■■ Duración: 2-4 horas

■

■■ Trigger: CloudWatch (latency p95 > 300ms)



■ ■ Auto-scaling group:

■ ■ ■ Min pods: 10

■ ■ ■ Max pods: 100

■ ■ ■ Target: CPU 60%, Memory 70%

■ ■ ■ Scale-up: +20 pods / 30s

■ ■ ■ Scale-down: -5 pods / 5min (cool-down)



■ ■ Cache pre-warming:

■ ■ ■ Predecir picos (partner calendarios)

■ ■ ■ Pre-load datos a Redis (1 hora antes)

■ ■ ■ Evitar cache stampede en t=0



■ ■ Database:

- ■ ■ RDS: auto-scaling hasta 100 connections
- ■ ■ Read replicas: usar para analytics queries
- ■ ■ Connection pooling: HikariCP (max 50 por pod)
-
- ■ Kafka:
- ■ ■ MSK: ya multi-broker, no requiere cambios
- ■ ■ Particiones: no aumentar durante pico (rebalance slow)
- ■ ■ Back-pressure: Kafka retiene, Core consume cuando puede
-
- ■ Validación: load test con Gatling
- ■ Ramp-up: 500 → 50,000 req/min en 5min
- ■ Sustain: 50,000 req/min por 30min
- ■ Expectation: p95 < 300ms, p99 < 500ms
- ■ Éxito: cero errores 5xx, zero dropped events

ESCENARIO 2: Evento paramétrico masivo (1M eventos/10min)

- ■ Origen: IoT/Telemetry (p.ej., tormenta, delays masivos)
- ■ Kafka Streams throughput: 1M eventos / 10min = 1,667 evt/s
-
- ■ Escalamiento:
- ■ ■ Kafka particiones: aumentar a 20 (si no done pre-scaling)
- ■ ■ Kafka Streams: scale up to 20 replicas
- ■ ■ Processor threads per pod: 2-4
- ■ ■ State store: RocksDB local, backup to S3
-
- ■ Back-pressure handling:
- ■ ■ Kafka consume slower than produce
- ■ ■ Kafka buffer: ~60s @ 1.6K evt/s = ~100K events
- ■ ■ Ledger service must keep up (parallel processing)
- ■ ■ If lag > threshold → alert ops, prepare for manual intervention
-
- ■ Database:
- ■ ■ Batch inserts for claims (no solo INSERT per event)
- ■ ■ Prepared statements, connection pooling

■ ■ ■ Expect: 1,667 claims/s for 10 min = 1M total

■

■ ■ Validación: Kafka StressTest

■ ■ Produce 1M events in 10min to test topic

■ ■ Streams consumer lag: should not exceed 30s

■ ■ Database INSERT rate: 1,667 op/s (achievable)

ESCENARIO 3: Reprocesamiento de perfiles (batch, 10M)

■ ■ Trigger: New Open Data source integrated

■ ■ Goal: Re-calculate all customer profiles < 2h

■ ■ Size: 10M customer profiles

■

■ ■ Fargate Batch Job:

■ ■ ■ Task definition: 4vCPU, 8GB memory

■ ■ ■ Concurrency: 100 tasks (400 vCPU total)

■ ■ ■ Duration: 10M profiles / (100 * 10 prof/task/min) = 10min

- ■ ■ Cost: ~\$0.50 (Spot instances)
-
- ■ Processing:
- ■ ■ Read: S3 parquet + db profiles (parallel)
- ■ ■ Enrich: Open Finance, Open Data APIs (batched)
- ■ ■ Aggregate: collect enrichments
- ■ ■ Write: bulk upsert to PostgreSQL
- ■ ■ Cache invalidation: Redis keys invalidated post-batch
-
- ■ Safeguards:
- ■ ■ Don't block online traffic
- ■ ■ CPU limit: 50% (leave headroom for online)
- ■ ■ DB connections: separate pool for batch
- ■ ■ If reprocessing fails, retain old profiles
- ■ ■ Retry: 3x with exponential backoff
-
- ■ Validation:
- ■ Sample: compare old vs new profiles (% changed)
- ■ Audit: log all changes
- ■ Approval: human review before "go-live"

5.2 Escalamiento Vertical

Componente	Phase 1	Phase 2	Phase 3
EKS Node	t3.xlarge (4vCPU)	t3.2xlarge (8vCPU)	r5.4xlarge (16vCPU, memory-opt)
RDS	db.t3.large (2vCPU, 8GB)	db.r5.xlarge (4vCPU, 32GB)	db.r5.4xlarge (16vCPU, 128GB)
Redis	cache.t3.medium (1GB)	cache.r6g.xlarge (26GB)	cache.r6g.2xlarge (52GB)
Kafka	broker.kafka.m5.large	broker.kafka.m5.2xlarge	broker.kafka.m5.4xlarge

6. API DESIGN (WEB, MÓVIL, PARTNERS)

6.1 Core API Contracts (Stable, único para todos)

OpenAPI 3.0.0

info:

title: Solventa Core API v1

version: 1.0.0

description: Shared contract for all clients

paths:

/quotations:

post:

operationId: createQuotation

summary: Request insurance quote

security:

- bearerAuth: []

- apiKeyAuth: [] # for partners

parameters:

- name: Accept-Version

in: header

schema:

enum: [v1, v2]

default: v1

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [coverage_types, amount]

properties:

partner_id:

type: string

description: Only for embedded integrations

customer_id:

type: string

format: uuid

description: If known customer, nullable for quotes

coverage_types:

type: array

items:

type: string

enum: [travel, device_protection, micro_life, parametric, mortgage_life]

amount:

type: number

format: decimal

minimum: 0.01

currency:

type: string

default: COP

enum: [COP, USD, MXN, CLP, PEN]

responses:

'200':

description: Quote calculated successfully

content:

application/json:

schema:

type: object

properties:

quote_id:

type: string

format: uuid

coverage_type:

type: string

amount:

type: number

currency:

type: string

premium_monthly:

type: number

description: Lowest-risk customer baseline

pricing_factors:

type: array

items:

type: object

properties:

factor_name:

type: string

impact_percentage:

type: number

explanation:

type: string

open_finance_factors:

type: object

nullable: true

description: If Open Finance consent given

properties:

payment_score:

type: number

minimum: 0

maximum: 100

risk_level:

type: string

enum: [low, medium, high]

valid_until:

type: string

format: date-time

description: Quote expiry (15 minutes)

terms_and_conditions:

type: string

format: url

'400':

description: Invalid input

'401':

description: Unauthorized

'429':

description: Rate limit exceeded

'503':

description: Service degraded (fallback quote returned)

/policies:

post:

operationId: issuePolicy

summary: Issue insurance policy (subscribe to quote)

security:

- bearerAuth: []

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [quote_id]

properties:

quote_id:

type: string

format: uuid

coverage_selections:

type: array

items:

type: object

properties:

coverage_type:

type: string

limit_amount:

type: number

open_finance_consent:

type: object

description: Consent for Open Finance/Open Data

properties:

consented:

type: boolean

data_types:

type: array

items:

type: string

enum: [payment_history, income, device_data, location, health_public]

revocation_uri:

type: string

format: uri

responses:

'201':

description: Policy issued successfully

content:

application/json:

schema:

type: object

properties:

policy_id:

type: string

format: uuid

policy_number:

type: string

status:

type: string
enum: [ISSUED, PAYMENT_PENDING, ACTIVE]
effective_date:
type: string
format: date
expiry_date:
type: string
format: date
premium_monthly:
type: number
policy_document_url:
type: string
format: uri
next_actions:
type: array
items:
type: string
'400':
description: Quote invalid/expired or validation error
'402':
description: Payment required (payment declined)
'422':
description: Unprocessable entity (underwriting decision)

/policies/{policy_id}:
get:
operationId: getPolicy
summary: Get policy details
security:
- bearerAuth: []
parameters:
- name: policy_id
in: path
required: true

schema:

type: string

format: uuid

responses:

'200':

description: Policy details

content:

application/json:

schema:

type: object

properties:

policy_id:

type: string

policy_number:

type: string

status:

type: string
enum: [ACTIVE, RENEWAL_PENDING, CANCELLED]

coverages:

type: array

items:

type: object

properties:

type:

type: string

limit:

type: number

deductible:

type: number

premium_monthly:

type: number

effective_date:

type: string

format: date

expiry_date:

type: string

format: date

renewal_options:

type: object

actions:

type: array

items:

type: string

enum: [modify_coverage, cancel, renew]

'404':

description: Policy not found or unauthorized

/claims:

post:

operationId: createClaim

summary: Report a claim

security:

- bearerAuth: []

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [policy_id, claim_type, description]

properties:

policy_id:

type: string

format: uuid

claim_type:

type: string

enum: [loss, theft, damage, illness, delay]
description:
type: string
maxLength: 1000
occurrence_date:
type: string
format: date
attachments:
type: array
items:
type: object
properties:
file_url:
type: string
format: uri
content_type:
type: string
enum: [image/jpeg, image/png, application/pdf, video/mp4]
responses:
'201':
description: Claim created
content:
application/json:
schema:
type: object
properties:
claim_id:
type: string
format: uuid
status:
type: string
enum: [SUBMITTED, UNDER_REVIEW, APPROVED, DENIED, PAID]
next_steps:

type: array

items:

type: string

estimated_review_time:

type: string

example: "24-48 hours"

/profiles/{customer_id}/mortgage-offer:

post:

operationId: getMortgageOffer

summary: Get personalized mortgage insurance offer

security:

- bearerAuth: []

parameters:

- name: customer_id

in: path

required: true

schema:
type: string
format: uuid
requestBody:
required: true
content:
application/json:
schema:
type: object
required: [credit_amount, term_months]
properties:
credit_amount:
type: number
minimum: 50000000 # 50M COP minimum
term_months:
type: integer
minimum: 60
maximum: 360
responses:
'200':
description: Personalized mortgage offer
content:
application/json:
schema:
type: object
properties:
offer_id:
type: string
format: uuid
premium_monthly:
type: number
coverage_amount:
type: number

terms:

type: string

validity:

type: string

format: date-time

pricing_explanation:

type: object

description: Why this price (explainability)

properties:

base_premium:

type: number

open_finance_discount:

type: number

nullable: true

age_factor:

type: number

location_factor:

type: number
health_factor:
type: number
nullable: true
final_premium:
type: number
quick_accept:
type: object
properties:
action_url:
type: string
format: uri
button_text:
type: string
'503':
description: Open Finance unavailable (fallback offer)
content:
application/json:
schema:
type: object
properties:
offer_id:
type: string
premium_monthly:
type: number
note:
type: string
example: "Standard rate (Open Finance data unavailable)"

components:
securitySchemes:
bearerAuth:
type: http
scheme: bearer

bearerFormat: JWT

description: Customer access token (24h validity)

apiKeyAuth:

type: apiKey

in: header

name: X-API-Key

description: Partner API key (for embedded integrations)

6.2 BFF Web Layer (Orchestration)

// Server: Node.js + Express + Fastify

// Purpose: Aggregate, transform, add web-specific logic

```
app.post('/web/wizard/quote-and-subscribe', async (req, res) => {
```

```
  const sessionId = req.session.id;
```

```
const userId = req.user.id;

// Step 1: Get quote from Core API
const quote = await coreAPI.post('/quotations', {
  customer_id: userId,
  coverage_types: req.body.coverageTypes,
  amount: req.body.amount
});

// Store in session for wizard state
req.session.quoteId = quote.quote_id;
req.session.quotedAt = new Date();

// Step 2: Enrich with web-specific data
const enriched = {
  quote,
  insuranceTerms: await fetchTermsAndConditions(quote.coverage_type),
  faq: await fetchFAQ(quote.coverage_type),
  similarProducts: await fetchRecommendations(quote),
  testimonials: await fetchTestimonials(quote.coverage_type),

  // Pre-fill form
  savedPaymentMethods: await getUserPaymentMethods(userId),
};

// Step 3: Log event for analytics
await analytics.event('quote_created', {
  user_id: userId,
  quote_id: quote.quote_id,
  coverage_type: quote.coverage_type,
  amount: quote.amount,
  timestamp: new Date(),
  device: req.device.type,
  referrer: req.referrer
});
```

```
res.json({
  status: 'success',
  data: enriched,
  wizard: {
    currentStep: 1,
    totalSteps: 4,
    steps: ['Quote', 'Review', 'Payment', 'Confirmation']
  }
});

});

app.post('/web/wizard/confirm-and-pay', async (req, res) => {
  const { quoteld } = req.session;
```

```
// Call Core API to issue policy
try {
  const policy = await coreAPI.post('/policies', {
    quote_id: quoteId,
    coverage_selections: req.body.selections,
    open_finance_consent: req.body.consent
  });

  // Store policy in session (for tracking)
  req.session.policyId = policy.policy_id;

  // Emit analytics event
  await analytics.event('policy_issued', {
    policy_id: policy.policy_id,
    user_id: req.user.id
  });

  res.json({
    status: 'success',
    data: {
      policyId: policy.policy_id,
      policyNumber: policy.policy_number,
      downloadUrl: policy.policy_document_url,
      nextSteps: policy.next_actions
    }
  });
} catch (error) {
  // Handle errors gracefully
  if (error.status === 402) {
    res.status(402).json({
      error: 'Payment declined',
      details: error.details,
      retryUrl: '/web/payment-retry'
    });
  }
}
```

```
} else if (error.status === 422) {  
  res.status(422).json({  
    error: 'Underwriting denied',  
    reason: error.underwritingReason,  
    appealUrl: '/web/appeal'  
  });  
}  
}  
});
```

6.3 BFF Mobile Layer (Offline-first, push)

// iOS/Swift implementation

```
class MobileAPIClient {
```

```

// Offline-first pattern
func fetchPolicies() async -> [Policy] {
// 1. Try remote API
if Reachability.isReachable {
let policies = try await remoteAPI.get("/mobile/policies")
// 2. Save locally
try localDatabase.save(policies)
return policies
} else {
// 3. Return cached
return try localDatabase.fetchPolicies()
}
}

// Offline claim reporting
func reportClaimOffline(policyId: String, description: String, photos: [UIImage]) async {
let claim = Claim(
policyId: policyId,
description: description,
status: .pending_sync,
photos: photos,
createdAt: Date()
)

// Save locally
try localDatabase.save(claim)

// If online, sync immediately
if Reachability.isReachable {
await syncPendingClaims()
} else {
// Mark for sync when back online
UserDefaults.pendingSyncCount = (UserDefaults.pendingSyncCount ?? 0) + 1
}
}

```

```
}
```

```
// Sync pending claims when back online
```

```
func syncPendingClaims() async {
```

```
let pending = try localDatabase.fetchClaimsPending()
```

```
for claim in pending {
```

```
do {
```

```
let response = try await remoteAPI.post("/mobile/claims/create-with-evidence", claim)
```

```
// Update local with server ID
```

```
claim.id = response.id
```

```
claim.status = .synced
```

```
try localDatabase.update(claim)
```

```

// Notify UI
NotificationCenter.default.post(name: NSNotification.Name("ClaimSynced"), object: response)

} catch {
// Retry later
print("Sync failed: \(error)")
}
}
}

// Biometric authentication
func authenticateWithBiometric() async -> AccessToken? {
let context = LAContext()
var error: NSError?

if context.canEvaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, error: &error) {
do {
try await context.evaluatePolicy(.deviceOwnerAuthenticationWithBiometrics,
localizedReason: "Access your Solventa account")

// Biometric succeeded, get access token
let token = try await remoteAPI.post("/mobile/biometric-auth",
{ biometric_verified: true })

// Save token securely
try Keychain.save(token, forKey: "access_token")

return token

} catch {
return nil
}
}

return nil
}

```

```
// Push notification handling

func handlePushNotification(_ payload: [AnyHashable: Any]) {
    guard let claimId = payload["claim_id"] as? String,
    let status = payload["status"] as? String else { return }

    // Sync claim state
    Task {
        let claim = try await remoteAPI.get("/claims/\(claimId)")
        try localDatabase.update(claim)

        // Show in-app notification
        showClaimStatusAlert(claim)
    }
}
```

```
}  
}
```

6.4 Partner API (Embedded)

=== PARTNER ONBOARDING ===

POST /partners/v1/onboarding

■ Request: { company_name, contact_email, use_case, api_usage_volume }

■ Response: { partner_id, api_key, sandbox_url, documentation_url }

■ SLA: approval < 1 working day

=== PARTNER INTEGRATION ===

1. Generate API Key

■ Rotated annually

■ OAuth2 scopes: [read_quotations, issue_policies]

■ Rate limit: 1,000 req/min, burst 2,000

2. Test in Sandbox

■ Full feature parity with production

■ Test data customers provided

■ Mock Kafka events

3. Go-live Checklist

■ Passed security scan

■ Load testing OK (p95 < 300ms)

■ Error handling implemented

■ Logging configured

■ SLA acknowledgment signed

■ Monitoring dashboard set up

=== PRODUCTION API EXAMPLE ===

// Partner: AviancaVenezuela.com

// Use case: Embed trip protection at booking

POST <https://api.solventa.co/partners/v1/quotations>

Headers:

■ ■ Authorization: Bearer sk_live_abc123...

■ ■ X-Idempotency-Key: a1b2c3d4... # optional

■ ■ Accept: application/json

Body:

```
{  
  "partner_id": "avianza-ve-001",  
  "customer_external_id": "cust_12345_avianza", # customer ref in their system  
  "coverage_type": "travel_insurance",
```

```
"origin": "Caracas (CCS)",
"destination": "Miami (MIA)",
"departure_date": "2026-02-15",
"return_date": "2026-02-22",
"trip_cost_usd": 450,
"travelers": [
{
"name": "Juan Rodríguez",
"age": 35,
"passport_country": "VE"
},
{
"name": "Maria Rodríguez",
"age": 33,
"passport_country": "VE"
}
]
```

Response (200):

```
{
"quote_id": "q_zx9w8v7u6t5",
"partner_ref": "avianza-ve-001",
"coverage_type": "travel_insurance",
"premium_usd": 18.50,
"per_traveler_usd": 9.25, # for display
"coverage": {
"medical_emergency": 50000,
"flight_delay": 200,
"luggage_loss": 1000,
"trip_cancellation": 450,
"covid_coverage": true
},
"valid_until": "2026-02-15T23:59:59Z",
```

```
"acceptance_url": "https://solventa.co/accept?q=q_zx9w8v7u6t5"
}
```

// Partner embeds in checkout:

// "Trip protection: 9.25 USD per person [ADD TO CART]"

POST https://api.solventa.co/partners/v1/policies

Headers:

■■ Authorization: Bearer sk_live_abc123...

■■ X-Idempotency-Key: avi_order_54321_001 # MUST be unique

■■ Accept: application/json

Body:

```
{
  "quote_id": "q_zx9w8v7u6t5",
```

```
"customer_external_id": "cust_12345_avianza",
"customer_email": "juan@example.com",
"customer_phone": "+58-416-123-4567",
"billing_address": {
  "street": "Av. Principal 123",
  "city": "Caracas",
  "country": "VE"
}
}
```

Response (201):

```
{
  "policy_id": "p_abc123def456",
  "policy_number": "AV-2026-001234",
  "status": "ACTIVE",
  "effective_date": "2026-02-15",
  "expiry_date": "2026-02-22",
  "premium_usd": 18.50,
  "policy_pdf_url": "https://documents.solventa.co/...",
  "reference_number": "AV-2026-001234",
  "customer_portal_url": "https://solventa.co/policies/p_abc123def456"
}
```

// Partner sends customer:

// ✓ Booking confirmation

// ✓ Trip insurance policy attached

// ✓ Customer can manage policy at: [customer_portal_url]

Error Handling:

429 Too Many Requests

■ ■ Retry-After: 60

■ ■ Reduce send rate or contact support

402 Payment Failed

- Reason: "customer_account_insufficient_balance"

- Partner should retry with alternative payment or show error to user

422 Underwriting Denied

- Reason: "high_risk_destination" or "age_exceeds_limit"

- Partner should present alternative coverage or remove from cart

Idempotency:

- Same X-Idempotency-Key within 24h → same response

- Prevents double-charging if POST retried

- Example: network timeout → auto-retry → idempotency prevents duplicate policy

7. DATOS Y CONSISTENCIA

7.1 Modelo de Datos (PostgreSQL)

-- Core tables

```
CREATE TABLE customers (  
  customer_id UUID PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  phone VARCHAR(20),  
  full_name VARCHAR(255),  
  
  -- PII encrypted at rest + in transit  
  document_id_encrypted BYTEA, -- DNI/passport, encrypted  
  document_type VARCHAR(20),  
  
  -- Open Finance consent  
  open_finance_consented BOOLEAN DEFAULT FALSE,  
  open_finance_consent_at TIMESTAMP,  
  open_finance_revoked_at TIMESTAMP NULL,  
  open_finance_data_types JSON, -- ['payment_history', 'income', ...]  
  
  -- Habeas data compliance  
  data_treatment_accepted BOOLEAN DEFAULT FALSE,  
  data_treatment_accepted_at TIMESTAMP,  
  
  kyc_status VARCHAR(20), -- PENDING, VERIFIED, FAILED  
  kyc_verified_at TIMESTAMP NULL,  
  kyc_provider_ref VARCHAR(255),  
  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  archived_at TIMESTAMP NULL  
);  
  
CREATE TABLE policies (  
  policy_id UUID PRIMARY KEY,
```

customer_id UUID NOT NULL REFERENCES customers,
partner_id VARCHAR(50) NULL, -- if embedded

coverage_type VARCHAR(50) NOT NULL, -- travel, device, life, mortgage, etc.
coverage_selections JSONB, -- { travel: 50000, delay: 200, ... }

premium_monthly DECIMAL(12,2),
premium_annual DECIMAL(12,2),
premium_currency VARCHAR(3) DEFAULT 'COP',

effective_date DATE NOT NULL,
expiry_date DATE NOT NULL,

```

renewal_date DATE NULL,

status VARCHAR(20), -- ISSUED, ACTIVE, RENEWAL_PENDING, CANCELLED, LAPSED

risk_profile JSONB, -- { risk_score: 0.45, factors: {...}, source: 'OPEN_FINANCE', ... }
rating_explanation TEXT, -- Why this price (for explainability)

policy_document_url VARCHAR(512),
e_signature_status VARCHAR(20), -- PENDING, SIGNED, FAILED

created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),

-- Audit
created_by_user_id UUID NULL,
underwriting_decision_at TIMESTAMP NULL,
underwriting_decided_by_user_id UUID NULL
);

-- Partitioned by year
CREATE TABLE policies_2026 PARTITION OF policies
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');

CREATE TABLE claims (
claim_id UUID PRIMARY KEY,
policy_id UUID NOT NULL REFERENCES policies,
customer_id UUID NOT NULL REFERENCES customers,

claim_type VARCHAR(50), -- loss, theft, damage, delay, parametric
claim_trigger_event VARCHAR(100) NULL, -- for parametric (flight_id, weather_event, etc.)

status VARCHAR(20), -- SUBMITTED, UNDER_REVIEW, APPROVED, DENIED, PAID, APPEALED

description TEXT,
occurrence_date DATE,
reported_date TIMESTAMP,

```

```
claimed_amount DECIMAL(12,2),
approved_amount DECIMAL(12,2) NULL,
paid_amount DECIMAL(12,2) NULL,

evaluator_notes TEXT NULL,
evaluation_completed_at TIMESTAMP NULL,

attachments JSONB, -- [{ file_id, type, url, uploaded_at }, ...]

created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);
```

```

CREATE TABLE transactions (
transaction_id UUID PRIMARY KEY,

-- Links
claim_id UUID NULL REFERENCES claims,
policy_id UUID NOT NULL REFERENCES policies,
customer_id UUID NOT NULL REFERENCES customers,

type VARCHAR(20), -- PREMIUM_CHARGE, CLAIM_PAYMENT, REFUND
direction VARCHAR(10), -- DEBIT, CREDIT

amount DECIMAL(12,2) NOT NULL,
currency VARCHAR(3) DEFAULT 'COP',

status VARCHAR(20), -- INITIATED, PROCESSING, COMPLETED, FAILED

payment_method VARCHAR(50), -- card, transfer, cash, etc.
payment_provider VARCHAR(50), -- provider name
provider_transaction_id VARCHAR(255), -- external ref

idempotency_key VARCHAR(255) UNIQUE NULL, -- for retry safety

created_at TIMESTAMP DEFAULT NOW(),
settled_at TIMESTAMP NULL

-- Immutable ledger
);

CREATE TABLE audit_log (
log_id BIGSERIAL PRIMARY KEY,

entity_type VARCHAR(50), -- policy, claim, customer, transaction
entity_id UUID NOT NULL,

action VARCHAR(50), -- CREATE, UPDATE, DELETE, DECISION, PAYMENT

old_values JSONB NULL,

```

```
new_values JSONB NULL,  
  
actor_type VARCHAR(20), -- SYSTEM, USER, PARTNER, BATCH  
actor_id UUID NULL,  
  
ip_address INET NULL,  
  
created_at TIMESTAMP DEFAULT NOW()  
  
-- Immutable, for compliance  
);  
  
-- Partitioned by week for performance
```

```

CREATE TABLE audit_log_2026_w01 PARTITION OF audit_log
FOR VALUES FROM ('2026-01-01') TO ('2026-01-08');

CREATE TABLE open_finance_consent (
  consent_id UUID PRIMARY KEY,
  customer_id UUID NOT NULL REFERENCES customers,

  consent_token VARCHAR(512),
  consented_at TIMESTAMP NOT NULL,
  revoked_at TIMESTAMP NULL,

  data_types TEXT[] DEFAULT '{payment_history,income}',

  created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_customers_email ON customers(email);
CREATE INDEX idx_policies_customer ON policies(customer_id);
CREATE INDEX idx_policies_status_date ON policies(status, effective_date);
CREATE INDEX idx_claims_policy ON claims(policy_id);
CREATE INDEX idx_claims_status_created ON claims(status, created_at);
CREATE INDEX idx_transactions_policy ON transactions(policy_id);
CREATE INDEX idx_transactions_created ON transactions(created_at);
CREATE INDEX idx_audit_entity ON audit_log(entity_type, entity_id);
CREATE INDEX idx_audit_created ON audit_log USING BRIN (created_at);

```

7.2 Estrategia de Caché (Redis)

Redis Cluster (3 master, 3 replica)

- Mode: cluster, not sentinel (HA built-in)
- Memory: 26GB (ElastiCache r6g.xlarge per node × 3 = 78GB total)
- Eviction: allkeys-lru (if memory exceeded, evict LRU items)
- Persistence: AOF (append-only file), sync every 1s
- Replication: async across AZs

DATA STRUCTURES:

1. Customer Profile Cache

Key: profile:{customer_id}

Type: HASH

TTL: 900s (15 min)

Size: ~2KB per customer

Hit rate: 85% (quota: 50M profiles $\times$ 0.85 = ~1M profiles cached)

Value:

{

risk_score: 0.45,

open_finance_signal: { payment_score: 85, income_stability: "high" },

```
open_data: { location: "Bogotá", risk_zone: "low" },
cached_at: 1705168200,
validity: 900
}
```

Invalidation:

- ■ TTL expiry (15 min)
- ■ Event: CustomerUpdated → HDEL
- ■ Event: OpenFinanceDataRefreshed → HDEL
- ■ Batch: reprocessing → DEL profile:*

2. Policy Cache

Key: policy:{policy_id}

Type: HASH

TTL: 3600s (1 hour)

Size: ~1KB per policy

Value:

```
{
customer_id: "uuid",
status: "ACTIVE",
coverage: { travel: 50000, delay: 200 },
expiry_date: "2027-01-15",
premium_monthly: 25.50
}
```

Invalidation:

- ■ PolicyUpdated event
- ■ PolicyCancelled event
- ■ TTL

3. Quote Request Cache

Key: quote:{quote_id}

Type: STRING (JSON)

TTL: 900s (15 min, lifetime of quote)

Value:

```
{  
  quote_id: "q_...",  
  customer_id: "uuid",  
  coverage_type: "travel",  
  amount: 50000,  
  premium_monthly: 25.50,  
  factors: [...],  
  created_at: 1705168200,  
  expires_at: 1705169100  
}
```

Pattern: write-through on quote creation

Read: cache-aside (miss → calculate → write)

4. Session Store

Key: session:{session_id}

Type: HASH

TTL: 86400s (web) or 604800s (mobile with remember-me)

Value:

```
{  
  user_id: "uuid",  
  email: "john@example.com",  
  device_type: "mobile",  
  last_activity: 1705168200,  
  selected_coverage: { travel: true, delay: false },  
  cart: { quote_id: "q_..." }  
}
```

Security: encrypted cookie, httpOnly, sameSite=strict

5. Bloom Filter (Fast Negative Check)

Key: bloom:fraud_blocklist

Type: BIT (Redisbloom module)

TTL: 300s (5 min, refresh from DB)

Size: ~10MB

Usage:

■ ■ Before accepting customer → BF.EXISTS bloom:fraud_blocklist customer_id

■ ■ If not exists → proceed

■ ■ If exists → check DB (false positive possible but rare)

■ ■ Benefit: avoid DB lookup 99.9% of time for legitimate customers

6. Reference Data (slowly changing)

Key: reference:{type}

Type: HASH

TTL: 86400s (24h, or event-invalidated)

Examples:

- ■ reference:coverage_types
- ■ reference:risk_factors
- ■ reference:underwriting_rules

Value: HGETALL returns { field1: value1, field2: value2, ... }

USAGE PATTERNS:

Pattern 1: Cache-Aside (Read-through)

- ■ GET key
- ■ If MISS:
 - ■ ■ Fetch from DB
 - ■ ■ SET key value EX ttl
 - ■ ■ Return value

- If HIT: return cached value

Pattern 2: Refresh-on-Write

- POST /policies → update DB
- DEL cache key (invalidate)
- Next read (cache miss) → repopulate
- Alternative: write-through (update cache immediately)

Pattern 3: Event-Driven Invalidation

- Event: PolicyIssued → Kafka
- Consumer: Cache invalidator
- DEL policy:{id}, DEL profile:{customer_id}
- Benefits: consistency, no stale data

MONITORING:

- Hit rate: target 85%
- Memory usage: alert if >90%
- Evictions: should be rare
- TTL accuracy: verify expiry times
- Latency p95: < 5ms (local cluster)

8. SEGURIDAD Y CUMPLIMIENTO

8.1 Data Security

ENCRYPTION IN TRANSIT:

- TLS 1.3: all traffic
- Certificate pinning: mobile apps
- HSTS: web (max-age=31536000)
- DNSSEC: domain

ENCRYPTION AT REST:

- RDS: AES-256 (AWS KMS)

■ ■ S3: AES-256 (AWS KMS)

■ ■ ElastiCache: in-transit encryption only (data is temporary)

■ ■ Backups: encrypted with separate KMS key

PII MASKING:

■ ■ Logs: no PII in logs (use UUIDs instead)

■ ■ Metrics: aggregate only (no customer IDs)

■ ■ Backups: encrypted key separate from DB key

■ ■ Exports: PII redacted or hashed

TOKENIZATION (for PII):

■ ■ Payment card: masked (****1234)

■ ■ Document ID: hashed (SHA-256)

- Phone: last 4 digits only in logs
- Storage: encrypted at application layer (double encryption)

8.2 Identity & Access Control

AUTHENTICATION:

- Customers:
 - Email + password (hashed with bcrypt, salt ≥ 12 rounds)
 - MFA: TOTP (Google Authenticator) or SMS
 - Biometric: Face ID / Touch ID (mobile)
 - Session: JWT (24h validity, refresh token)
- Operators (internal):
 - SSO: SAML / OIDC (Microsoft Entra)
 - MFA: mandatory (hardware key or TOTP)
 - Session: 8h (logout on idle 30 min)
- Partners:
 - OAuth2: Authorization Code flow
 - API Key: rotated annually, encrypted storage
 - Rate limiting: per partner, per endpoint

AUTHORIZATION (RBAC):

- Customers:
 - Scope: own policies, own claims
 - Actions: view, create claim, modify coverage
 - Enforcement: customer_id in JWT, check on each request
- Operators:
 - Roles: Advisor, Underwriter, Claims Manager, Compliance Officer, Admin
 - Permissions: mapped to role, enforced per endpoint
 - Audit: all actions logged with actor_id
- Partners:

- ■ Scopes: read_quotations, issue_policies
- ■ Isolation: can only see own partner_id data
- ■ Audit: API calls logged

OPEN FINANCE CONSENT:

- ■ Verification:
 - ■ ■ Customer explicitly grants permission in UI
 - ■ ■ Confirmation email sent (CTA: confirm/deny)
 - ■ ■ Stored in open_finance_consent table
 - ■ ■ Timeout: 72h to confirm
- ■ Usage:
 - ■ ■ Only for pricing/risk assessment (stated purpose)
 - ■ ■ Audit trail: what data, when, by whom
 - ■ ■ Minimization: only necessary fields

■

■ ■ Revocation:

■ ■ ■ Customer can revoke anytime (UI button)

■ ■ ■ Request sent to Open Finance provider within 5 min

■ ■ ■ Core stops calling provider APIs

■ ■ ■ Cached data invalidated immediately

■

■ ■ Compliance:

■ ■ SFC audit trail: all consents logged

■ ■ Habeas data: customer request to delete → full compliance

■ ■ Reporting: monthly consent audit to DPA (data protection authority)

8.3 Fraud Detection

REAL-TIME RULES:

■ ■ Amount threshold: claim > \$5,000 → manual review

■ ■ Frequency: > 3 claims in 30 days → flag for investigation

■ ■ Pattern matching:

■ ■ ■ Same beneficiary, different customers → fraud signal

■ ■ ■ Claim within 24h of policy issue → risk flag

■ ■ ■ High-value claims outside policy limit → immediate deny

■ ■ ■ Multiple claims same event → coordinate

■

■ ■ Velocity check:

■ ■ ■ Policy issued → how many claims expected?

■ ■ ■ If `claims_count / expected > 2x` → investigation

■ ■ ■ Confidence score: 0-100

■

■ ■ Scoring:

■ ■ Risk score per claim (0-100)

■ ■ < 30: auto-approve (no review)

■ ■ 30-70: manual review (human decides)

■ ■ > 70: auto-deny (with appeal option)

MACHINE LEARNING (Future):

- ■ Training data: historical fraud cases (labeled)
- ■ Features: claim amount, policy age, frequency, customer history
- ■ Model: XGBoost (explainable)
- ■ Prediction: fraud probability per claim
- ■ Action: trigger investigation if $P > \text{threshold}$

MONITORING:

- ■ Fraud rate: target $< 0.5\%$ of claims
- ■ False positive rate: $< 5\%$ (too many delays)
- ■ Investigation turnaround: < 5 days
- ■ Appeal success rate: track (if $> 10\%$, rules too strict)

PREVENTIVE MEASURES:

- ■ KYC: thorough identity verification at onboarding

- ■ Habeas data: check against known fraudsters lists (external)
- ■ Velocity limiting: rate-limit quote requests per IP
- ■ Device fingerprinting: detect suspicious devices/IPs

8.4 Compliance Frameworks

LEY 1581 (HABEAS DATA - Protección de datos personales):

- ■ Consent: explicit, informed, prior (handled ✓)
- ■ Purpose limitation: only use for stated purpose
- ■ Data minimization: collect only necessary
- ■ Accuracy: keep PII current
- ■ Retention: delete after purpose fulfilled (typically 3 years)
- ■ Right to access: API endpoint GET /me/data
- ■ Right to rectification: API endpoint PATCH /me/data
- ■ Right to erasure: API endpoint DELETE /me (anonymize, not delete)
- ■ Accountability: DPA reports, audit trail

DECRETO 1297 / CIRCULAR 004 (OPEN FINANCE):

- ■ Consent framework: SFC-approved template
- ■ Revocation: effective within 5 min
- ■ Data minimization: only necessary fields
- ■ Audit trail: what was requested, approved, used
- ■ Provider compliance: verify SFC authorization
- ■ Reporting: monthly to SFC on data usage

PCI-DSS (Payment Card Industry):

- ■ Card data: never stored (tokenized via provider)
- ■ Transmission: TLS only, P2PE (Provider-managed)
- ■ Environment: PCI-compliant AWS account (separate)
- ■ Audit: annual compliance certification
- ■ Encryption: key stored separately, rotated annually

OPEN BANKING API SECURITY:

- ■ Authentication: mutual TLS (mTLS)
- ■ Signing: JWS (JSON Web Signature) for non-repudiation

- ■ Validation: signature verification on all payloads
- ■ Timestamp: prevent replay attacks (within 5 min window)
- ■ Encryption: JWE (JSON Web Encryption) for sensitive data

REPORTING:

- ■ SFC:
 - ■ ■ Monthly: policies issued, claims paid, complaints
 - ■ ■ Quarterly: risk metrics, capital ratios
 - ■ ■ Annual: audited financial statements
 - ■ ■ Ad-hoc: significant events (data breach, operational failure)
- ■ ■
- ■ Superintendencia de Vigilancia y Seguridad Privada:
 - ■ ■ Risk reports if armed response services used
- ■ ■

■ ■ Defensoría del Consumidor:

■ ■ Complaints: must respond < 30 days

9. RESILIENCIA Y DISPONIBILIDAD

9.1 Fault Tolerance Patterns

CIRCUIT BREAKER (Open Finance / Payment API):

State: CLOSED (normal)

■ ■ Requests: pass through

■ ■ Failures: count

■ ■ Threshold: if failure_rate > 50% in 1min → OPEN

State: OPEN (tripped)

■ ■ Requests: fast-fail immediately

■ ■ Response: cached data or fallback offer

■ ■ Duration: 60s → HALF_OPEN

State: HALF_OPEN (testing recovery)

■ ■ Requests: sample (1 per 10s)

■ ■ If success: → CLOSED

■ ■ If failure: → OPEN (reset timer)

■ ■ Timeout: 30s total in HALF_OPEN

Implementation:

```
@CircuitBreaker(
```

```
failureThreshold = 50, // 50% failure rate
```

```
delay = 60000, // 60s before HALF_OPEN
```

```
successThreshold = 2, // 2 success → CLOSED
```

```
failureRatioOnTimeout = true // count timeout as failure
```

```
)
```

```
@Timeout(value = 120, unit = ChronoUnit.MILLIS)
```

```
public Quote getOpenFinanceSignal(String customerId) {  
    // Call external API  
    // If circuit open → return cached profile or fallback  
}
```

TIMEOUT PATTERN:

External API call:

- ■ Timeout: 120ms (hard limit)
- ■ Retry: 1x after 50ms
- ■ If still timeout → use cached data (if age < 5 min)
- ■ If no cache → use default factors (conservative pricing)
- ■ Log: "OpenFinance timeout, used cache"

BULKHEAD PATTERN (Thread isolation):

Solventa Core process:

- Main thread pool: 100 threads (request handling)
- Open Finance pool: 20 threads (external calls, isolated)
- Database pool: 50 threads (DB operations)
- Batch pool: 10 threads (async/batch jobs)

Benefit: if Open Finance stalls, doesn't affect other services

RETRY WITH BACKOFF:

```
public <T> T retryWithBackoff(Callable<T> task, int maxAttempts) {
    int attempt = 0;
    while (attempt < maxAttempts) {
        try {
            return task.call();
        } catch (Exception e) {
            if (attempt == maxAttempts - 1) throw e;

            long backoffMs = Math.min(
                1000 * (long) Math.pow(2, attempt), // exponential
                30000 // max 30s
            );
            Thread.sleep(backoffMs + random(0, 1000)); // jitter
            attempt++;
        }
    }
}
```

FALLBACK STRATEGY:

Scenario: Open Finance API returns error

Option 1: Cache fallback

- ■ Has cached profile < 5 min old?
- ■ Yes → use it (might be slightly stale)
- ■ No → proceed to Option 2

Option 2: Default factors

- ■ Use conservative (default) risk factors
- ■ Price slightly higher than optimal
- ■ But user can still get quote
- ■ Log event for analysis

Option 3: Manual review

- ■ Quote saved as "pending_review"
- ■ Operator looks manually (hours delay)
- ■ Only if high value / suspicious

9.2 High Availability Architecture

ACTIVE-ACTIVE MULTI-REGION (Future - Phase 2):

Region: us-east-1 (Bogotá logical)

- ■ EKS cluster (primary)
- ■ RDS PostgreSQL (writer)
- ■ Redis cache
- ■ Kafka cluster

Region: us-west-2 (standby for read, can become primary)

- ■ EKS cluster (standby, low capacity)
- ■ RDS PostgreSQL read replica
- ■ Redis cache (replication from primary)
- ■ Kafka replication

Failover flow:

- ■ Monitor: primary region latency/errors
- ■ Trigger: if p95 > 500ms or error rate > 5%
- ■ Promote: read replica → writer
- ■ Redirect: DNS to us-west-2
- ■ Notify: ops team, customers (no SLA breach if < 5 min)
- ■ Recovery: restore primary, sync data, revert DNS

RTO: < 5 minutes

RPO: < 30 seconds (async replication)

WITHIN-REGION HA:

Multi-AZ deployment:

- ■ us-east-1a: EKS node, Kafka broker, RDS replica
- ■ us-east-1b: EKS node, Kafka broker, RDS master
- ■ us-east-1c: EKS node, Kafka broker, Kafka replica
- ■ SLA: if any AZ fails → service continues

RDS failover:

- ■ Master in 1b fails
- ■ Automatic: promote 1a replica to master (30s)
- ■ DNS updated: connections redirected
- ■ New replica created in 1c (async)
- ■ Zero downtime for persistent connections (connection pooling handles)

EKS multi-AZ:

- ■ Pods spread across 1a, 1b, 1c
- ■ Service load balancer: routes around failed pods
- ■ If node fails: pods rescheduled to healthy nodes (< 30s)
- ■ Minimum replicas: 3 (for 99.9% availability)

9.3 Monitoring & Alerting

METRICS (Prometheus):

- ■ Request latency: p50, p95, p99 per endpoint
- ■ Error rate: 5xx, 4xx per endpoint
- ■ Database latency: query duration, connection pool usage
- ■ Cache hit rate: % of requests served from cache
- ■ Queue depth: Kafka lag per consumer group
- ■ Resource usage: CPU, memory, disk per pod/node
- ■ Business metrics: quotes/min, policies/min, claims/min

DASHBOARDS (Grafana):

- ■ Overview: system health, key SLIs
- ■ Latency: p95 quotation, p95 claim, p95 payment
- ■ Errors: error rate, error types, top failing endpoints
- ■ Dependencies: Open Finance availability, payment provider health
- ■ Database: query performance, index usage, replication lag
- ■ Events: Kafka throughput, consumer lag, dead letters

ALERTING (PagerDuty):

■ ■ Critical:

- ■ ■ Error rate > 5% for 1 min → page on-call
- ■ ■ p95 latency > 500ms for 5 min → page
- ■ ■ Database replication lag > 60s → page
- ■ ■ Kafka consumer lag > 100k messages → page
- ■ ■ Payment provider down → page (critical path)

■

■ ■ Warning:

- ■ ■ Error rate 2-5% for 5 min → notify in Slack
- ■ ■ p95 latency 300-500ms for 5 min → notify
- ■ ■ Cache hit rate < 70% → investigate
- ■ ■ Memory usage > 80% → prepare for scale
- ■ ■ Disk usage > 80% → clean up logs

■

- Info (Slack only):
- Deployment succeeded/failed
- Database backup completed
- Scheduled maintenance windows
- New provider integrated

10. PLAN DE EXPERIMENTOS

10.1 Validación de Atributos de Calidad

EXPERIMENTO 1: Latencia - Cotización Embebida



Objetivo: Validar $p95 < 250\text{ms}$, $p99 < 500\text{ms}$

Hipótesis: "Con Core modular + Redis + Open Finance timeout 120ms, podemos satisfacer latencia en pico de 50k cot/min"

Setup:

- ■ Test environment: mirror prod (EKS, RDS, Redis, MSK)
- ■ Load generator: Gatling, 50k requests/min ramp
- ■ Duration: 10 min sustained
- ■ Monitoring: Prometheus + Grafana (real-time)
- ■ Network: AWS, same AZ (representative of prod)

Scenario 1: Nominal case (cache hit, online provider)

- ■ Request: customer has cached profile, Open Finance online
- ■ Expected: $p95 < 250\text{ms}$
- ■ Load: 50k cot/min
- ■ Result: PASS if $p95 < 250\text{ms}$ for all 10 min

Scenario 2: Cache miss (profile expired, recompute)

- ■ Request: customer profile cache expired
- ■ Expected: $p95 < 300\text{ms}$ (slight penalty)
- ■ Load: 50k cot/min
- ■ Result: PASS if $p95 < 300\text{ms}$

Scenario 3: Open Finance degraded (120ms timeout)

- ■ Request: Open Finance provider slow (150ms response)
- ■ Expected: timeout @ 120ms, fallback to default factors, $p95 < 200\text{ms}$
- ■ Load: 50k cot/min
- ■ Result: PASS if $p95 < 250\text{ms}$ (timeout respected)

Scenario 4: Network latency spike

- ■ Simulate: 200ms network RTT (bad WiFi, satellite)
- ■ Expected: timeouts trigger, fallback used, $p95 < 300\text{ms}$
- ■ Load: 50k cot/min

■■ Result: PASS if no 5xx errors, cache fallback works

Success criteria:

■ All scenarios pass p95 target

■ Zero 5xx errors

- Cache hit rate > 80%

■ Open Finance timeout respected ($\leq 120\text{ms}$)

■ Memory usage < 80%

EXPERIMENTO 2: Escalabilidad - Evento Paramétrico Masivo



Objetivo: Validar absorción de 1M eventos/10min

Hipótesis: "Kafka Streams + 20 particiones + Ledger service"

puede procesar 1,667 evt/s sin data loss"

Setup:

- ■ Kafka: 5 brokers, 20 partitions per topic
- ■ Streams: 20 stream processor tasks (parallelism = 20)
- ■ Ledger service: 10 pods, 2 threads each = 20 parallel
- ■ Database: RDS t3.2xlarge with connection pooling
- ■ Load: Kafka producer → 1,667 events/sec for 10 min

Scenario 1: Sustained load (1.6K evt/sec)

- ■ Produce: 1,667 events/sec continuously
- ■ Expect: consumer lag stays < 30s
- ■ DB: 1,667 inserts/sec sustained
- ■ Result: PASS if lag < 30s end-to-end, zero data loss

Scenario 2: Spike (3.3K evt/sec for 60s, then 1.6K)

- ■ Produce: ramp to 3.3K/sec, hold 60s, back to 1.6K
- ■ Expect: lag spike but recover within 2 min
- ■ DB: burst handling, connection pool exhaustion?
- ■ Result: PASS if lag recovers < 2 min, no errors

Scenario 3: Back-pressure (producer slower than consumer can process)

- ■ Simulate: DB slow (3 sec query latency)
- ■ Expect: Kafka buffer accumulates, no events dropped
- ■ Recovery: once DB recovers, catch up
- ■ Result: PASS if zero data loss, no exceptions

Success criteria:

- ■ Zero events dropped (compare produced vs DB inserts)
- ■ Consumer lag < 30s @ nominal load
- ■ Lag recovery < 2 min after spike
- ■ Database insert throughput > 1,700 ops/sec
- ■ No connection pool exhaustion errors

EXPERIMENTO 3: Disponibilidad - Failover Multi-AZ



Objetivo: Validar $RTO \leq 10\text{min}$, $RPO \leq 30\text{s}$ en caída de AZ

Hipótesis: "Multi-AZ RDS + DNS failover mantiene disponibilidad $\geq 99.97\%$ "

Setup:

■ ■ 3 AZs: us-east-1a, 1b, 1c (each with node + pod)

■ ■ RDS: Master in 1b, replica in 1a

■ ■ Kafka: 3 brokers across AZs

■ ■ Load: continuous traffic (1,000 cot/min)

■ ■ Monitoring: Prometheus, downtime tracking

Scenario 1: AZ 1a fails (network partition)

■ ■ Impact: -1 EKS node, -1 Kafka broker, -1 RDS replica

■ ■ Expected: remaining pods in 1b,1c handle traffic

■ ■ RDS: still has master in 1b (unaffected)

- Kafka: rebalance to 2 brokers (minor lag spike)
- Result: PASS if downtime < 30s, zero 5xx errors

Scenario 2: AZ 1b fails (RDS master region)

- Impact: -1 EKS node, -1 Kafka broker, -RDS master
- Expected: RDS automatic failover (30s) to replica in 1a
- DNS: updated to master in 1a
- Kafka: rebalance to 1a, 1c (leader election ~20s)
- Result: PASS if RTO $\leq$ 60s (30s RDS + 30s Kafka rebalance)

Scenario 3: Network partition (not full AZ failure)

- ■ Simulate: 1a ↔ 1b network latency > 1s, loss 50%
- ■ Expected: circuit breaker trips for cross-AZ calls
- ■ Fallback: local cache used
- ■ Result: PASS if latency < 300ms, error rate < 1%

Success criteria:

- ■ RTO ≤ 10 min for any single AZ failure
- ■ RPO ≤ 30 s (RDS replication async, data loss at most 30s)
- ■ Availability $\geq 99.9\%$ (≤ 1 hour downtime/month)
- ■ Zero data corruption
- ■ Failback successful (original AZ recovered, switched back)

EXPERIMENTO 4: Seguridad - Cumplimiento HABEAS DATA



Objetivo: Validar que PII no se expone, auditoría completa

Hipótesis: "Con encriptación + logging + consent management, cumplimos Ley 1581 y SFC requirements"

Setup:

- Test data: 1,000 synthetic customers + policies
- Compliance scan: automated (PII detection in logs/DB)
- Manual review: sample audit trail
- Third-party: PCI-DSS compliance report

Scenario 1: PII in logs (prevention)

- ■ Generate: 1M log entries from different modules
- ■ Scan: regex for email, phone, ID, credit card
- ■ Expected: zero matches (all should be UUID/hash)
- ■ Result: PASS if scan returns 0 PII violations

Scenario 2: Customer data export (GDPR/habeas data right)

- ■ Request: /me/data (full customer data export)
- ■ Expected: all PII included, encrypted, audit logged
- ■ Verify: email received, data matches DB, timestamp correct
- ■ Result: PASS if export complete < 1 min, audit trail present

Scenario 3: Customer data deletion (right to erasure)

- ■ Request: DELETE /me (anonymize all customer data)

- Expected: PII removed, policies retained (audit need)
- Verify: customer_id anonymized, audit log preserved
- Result: PASS if deletion complete < 5 min, full audit trail

Scenario 4: Open Finance consent verification

- Request: customer grants Open Finance consent
- Expected: consent token generated, email confirmation sent
- Verify: audit log shows consent grant with timestamp
- Request: revoke consent
- Expected: consent revoked < 5 min, API stops calling provider
- Verify: audit log shows revocation, timestamp recorded
- Result: PASS if all timestamps audit-logged, revocation effective

Success criteria:

- ■ Zero PII in logs
- ■ Data export: complete < 1 min, encrypted
- ■ Data deletion: < 5 min, audit-logged
- ■ Open Finance: consent verified, revocation < 5 min
- ■ Audit trail: 100% of sensitive actions logged
- ■ SFC compliance: external audit passes

EXPERIMENTO 5: Modificabilidad - Nuevo Ramo (Mascotas)



Objetivo: Validar que agregar nuevo ramo toma ≤ 2 semanas

Hipótesis: "Modular Core permite agregar coverage type sin tocar núcleo"

Setup:

- ■ New coverage_type: "pet_insurance"
- ■ New features:
- ■ ■ Pricing: deductible, annual limit
- ■ ■ Claims: vet visit, emergency, surgery
- ■ ■ Underwriting: pet age/breed validation
- ■ ■ Coverage limits: \$100-\$10k annual
-

■ ■ Team: 1 backend eng, 1 frontend, 1 QA

■ ■ Tracking: Jira sprint (2 weeks)

Sprint timeline:

Week 1:

■ ■ Day 1-2: Add risk factors for pets (age, breed, health)

■ ■ → Code: add new enum CoverageType.PET_INSURANCE

■ ■ → DB: migrate to add pet-specific columns

■ ■

■ ■ Day 3-4: Implement pricing rules for pets

■ ■ → RatingEngine: new branch for PET_INSURANCE

■ ■ → Rules: age-based premiums, breed multipliers

■ ■

■ ■ Day 5: Unit tests, peer review

■■ → Target: 80% code coverage for pet pricing

Week 2:

■ Day 1-2: Underwriting rules, claims workflow

■ ■ → UnderwritingEngine: new rules for pet eligibility

■■ → ClaimsService: pet-specific fields (vet details)



■ Day 3: Integration tests, end-to-end flow

■■ → Test: quote → subscribe → claim payment

■ Day 4: Load test, security review

■■ → Latency: verify p95 < 250ms still met

■■ → Security: no data leaks, consent still validated



■ Day 5: Deploy to staging, manual testing, docs

Success criteria:

■ ■ ≤ 2 weeks elapsed

■■ New coverage_type available in production

■■ Latency: p95 still < 250ms

- No core logic modified (all in modules)

■ ■ Audit trail: new rules logged

- Marketing: can advertise new coverage within 3 days

EXPERIMENTO 6: Integración - Nuevo Socio Embebido (Marketplace)

[illegible]

Objetivo: Validar que agregar partner toma ≤ 1 semana

Hipótesis: "Partner API + OAuth2 permite onboarding autónomo"

Setup:

- ■ New partner: "MercadoLibreMX" (e-commerce)

■ ■ Use case: coverage for purchased items during shipping

■ Integration: embed quote widget in checkout

- Team: partner eng + ops (Solventa 1 person support)

Timeline:

Day 1:

■■ Request received, check compliance (AML, SFC)

■■ → Result: approved

■■ → Provide: sandbox credentials, API docs

Day 2:

■■ Partner: implement OAuth2 flow

■■ → Solventa: provide token endpoint

■■ → Partner: test in sandbox (quote API)

Day 3-4:

■■ Partner: integrate quote widget in MX checkout

- ■ → Test: price quotes, add to cart
- ■ → Review: margins, FX rates, local regulations

Day 5:

- ■ Load test: simulate partner traffic (1k users/hour)
- ■ → Solventa: verify p95 < 250ms, no errors
- ■ → Partner: sign SLA, API key generated

Day 6-7:

- ■ Go-live in production
- ■ → Monitor: first 24h, watch error rate
- ■ → Support: escalation channel (Slack)
- ■ → Success: first policies issued from partner

Success criteria:

- ■ ≤ 1 week elapsed
- ■ Partner can onboard without Solventa dev resources
- ■ Latency: p95 < 250ms from partner perspective
- ■ Error rate: < 0.1%
- ■ First policies issued within 24h go-live
- ■ Partner self-service: can manage API keys, view dashboard

CONCLUSIÓN

Este **System Design de Solventa** propone una arquitectura **modular, escalable, resiliente y compatible** con cumplimiento regulatorio, balanceando los 6 atributos de calidad mediante decisiones conscientes sobre trade-offs:

| Atributo | Enfoque |

|-----|-----|

| **Latencia** | Monolito en Core + Redis + timeouts + fallbacks |

| **Escalabilidad** | HPA en EKS, event stream, data locality |

| **Disponibilidad** | Multi-AZ, RDS failover, circuit breakers |

| **Seguridad** | Encryption, RBAC, audit trail, compliance-by-design |

| **Modificabilidad** | Módulos bien delimitados, no requiere refactor para nuevos ramos |

| **Integración** | BFF por canal, APIs versionadas, adapters estándar |

Los **experimentos** validan cada decisión con evidencia medible antes de comprometerse.

Versión: 1.0 | **Fecha:** 2026-01-15 | **Estado:** Diseño Preliminar

Próximos pasos: Presentar al equipo, iterar feedback, comenzar fase spike (implementación POC)